Assignment 2

Polygon Testnet Deployment

Name: Sohan Metil
Course: B.Tech Computer Engineering
Subject: Blockchain Technology
Date: April 20, 2026

1. Objective

The goal of this assignment is to deploy a Solidity-based smart contract onto the Polygon Amoy Testnet using the Hardhat development framework. Polygon is a Layer 2 scaling solution for Ethereum that offers faster transaction speeds and significantly lower gas fees while maintaining compatibility with the Ethereum Virtual Machine (EVM).

By the end of this exercise, the contract is live on a public testnet and its deployment is independently verifiable through the Polygon Amoy block explorer.

2. Smart Contract

The same SimpleStorage contract from Assignment 1 is reused here, with a minor rename of the state variable and event for clarity. The contract stores a single unsigned integer value on-chain and provides read and write access through two public functions.

2.1 contract.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {

    uint256 private value;

    event ValueChanged(uint256 newValue);

    function setValue(uint256 newValue) public {
        value = newValue;
        emit ValueChanged(newValue);
    }

    function getValue() public view returns (uint256) {
        return value;
    }
}
```

2.2 Contract Description

The contract contains two core functions. setValue() accepts an unsigned integer, writes it to the private state variable, and emits a ValueChanged event so external systems can track updates. getValue() is a read-only view function that returns the current stored value without incurring any gas cost.

3. Network Details

The Polygon Amoy Testnet was selected for deployment. The following network configuration was used throughout this assignment:

Parameter	Value
Network Name	`Polygon Amoy Testnet`
RPC URL	`https://rpc-amoy.polygon.technology`
Chain ID	`80002`
Currency Symbol	`POL`
Block Explorer	`https://amoy.polygonscan.com`

4. Deployment Script

The following Hardhat deployment script was used to compile and publish the contract to the Polygon Amoy Testnet. The script leverages the ethers.js library bundled with Hardhat to interact with the network.

4.1 scripts/deploy.js

```
const hre = require("hardhat");

async function main() {
  const SimpleStorage = await hre.ethers.getContractFactory("SimpleStorage");
  const contract = await SimpleStorage.deploy();

  await contract.waitForDeployment();

  console.log("Contract deployed to:", await contract.getAddress());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

5. Steps to Deploy on Polygon Amoy Testnet

Step 1 — Initialize the Node.js Project

```
npm init -y
```

Step 2 — Install Hardhat

```
npm install --save-dev hardhat
```

Step 3 — Initialize Hardhat Project

```
npx hardhat
```

Select Create a JavaScript project when prompted and accept the default options.

Step 4 — Install Additional Dependencies

```
npm install @nomicfoundation/hardhat-toolbox dotenv
```

Step 5 — Configure the .env File

Create a file named .env in the project root and add your wallet private key:

```
PRIVATE_KEY=your_wallet_private_key_here
```

Important: Never share or commit this file. Add .env to your .gitignore file to prevent accidental exposure.

Step 6 — Update hardhat.config.js

Modify the Hardhat configuration file to include the Polygon Amoy network:

```
require("@nomicfoundation/hardhat-toolbox");
require("dotenv").config();

module.exports = {
  solidity: "0.8.0",
  networks: {
    amoy: {
      url: "https://rpc-amoy.polygon.technology",
      accounts: [process.env.PRIVATE_KEY],
      chainId: 80002,
    },
  },
};
```

Step 7 — Compile the Contract

```
npx hardhat compile
```

A successful compilation generates the contract ABI and bytecode under the artifacts/ directory.

Step 8 — Deploy the Contract

```
npx hardhat run scripts/deploy.js --network amoy
```

On success, the terminal displays the deployed contract address on the Polygon Amoy Testnet.

6. Deployed Contract Address

The smart contract was deployed successfully. The contract address on the Polygon Amoy Testnet is:

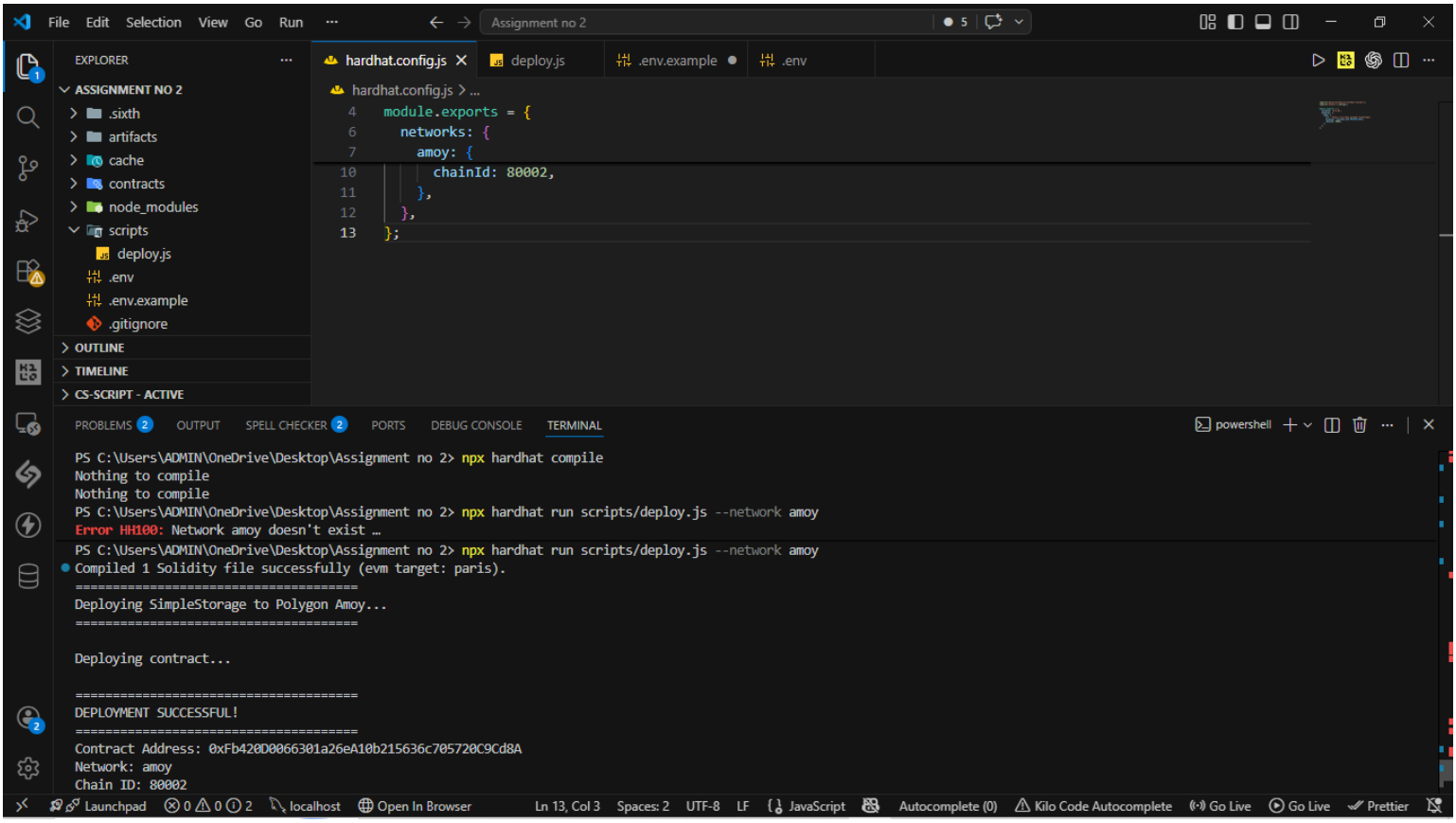
```
0xFb420D0066301a26eA10b215636c705720C9Cd8A
```

This address can be looked up on the Polygon Amoy block explorer at https://amoy.polygonscan.com to verify the contract creation transaction, bytecode, and all subsequent interactions.

7. Screenshots

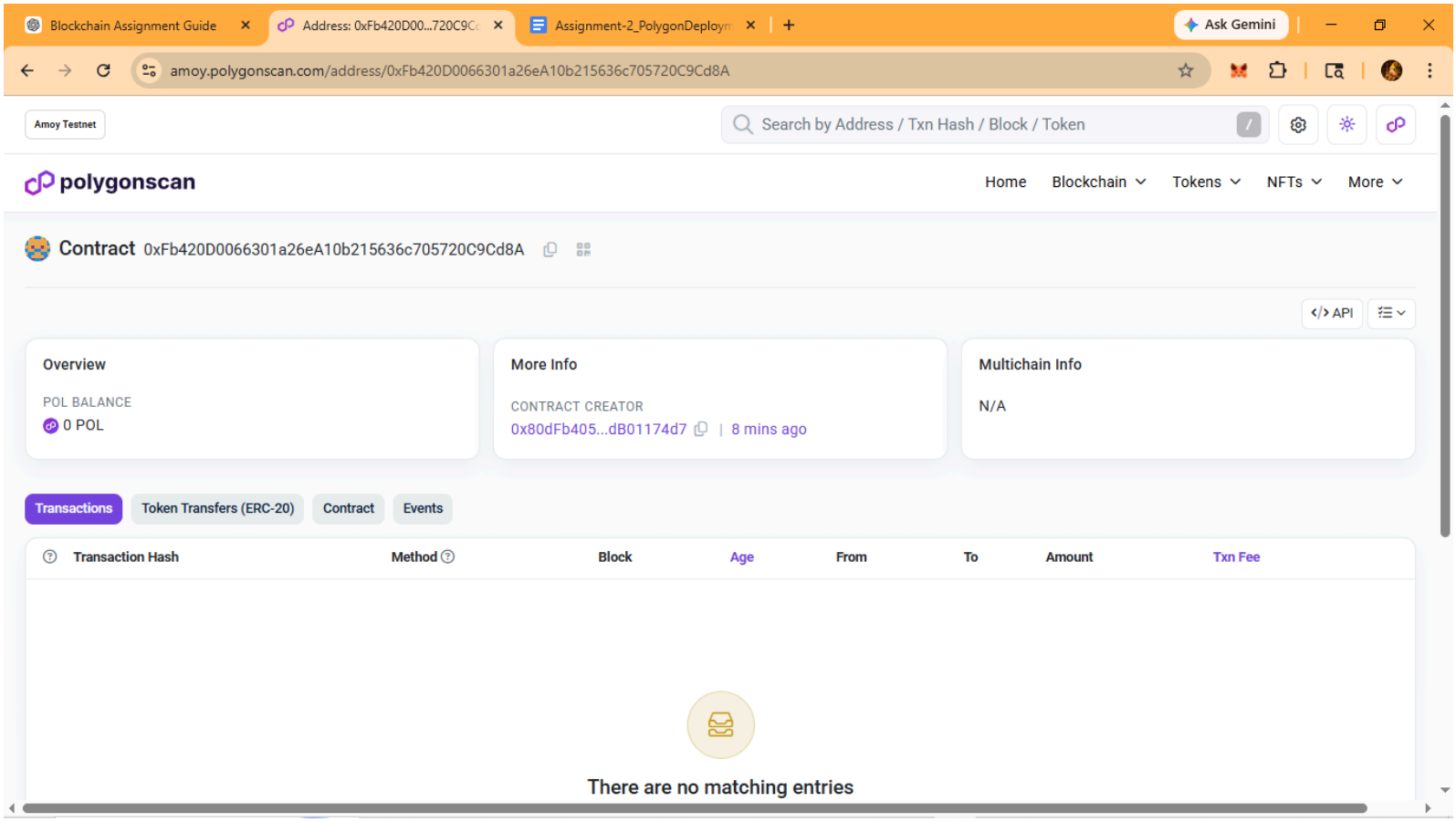
7.1 Deployment Success — Terminal Output

The following screenshot shows the terminal output after running the Hardhat deployment command, including the confirmation message and the generated contract address.



7.2 Transaction on Polygon Amoy Explorer

The screenshot below shows the contract address page on PolygonScan (Amoy), confirming the contract creation transaction, network details, and on-chain status.



8. Testing

To verify the deployed contract functions correctly, the contract was tested using the Remix IDE by connecting it to the Injected Provider (MetaMask) pointed at the Amoy testnet.

8.1 Test Sequence

1. Connected Remix IDE to the Polygon Amoy Testnet via MetaMask.
2. Loaded the deployed contract using its address: 0xFb420D0066301a26eA10b215636c705720C9Cd8A.
3. Called getValue() — returned 0, confirming the initial default state.
4. Called setValue(100) — transaction was broadcast to the Amoy testnet and confirmed.
5. Called getValue() again — returned 100, verifying the on-chain state update.

9. Conclusion

The SimpleStorage contract was deployed successfully on the Polygon Amoy Testnet using the Hardhat development framework. The deployment was verified through the PolygonScan block explorer, confirming that the contract is live and accessible on a public blockchain network.

This assignment demonstrated the process of configuring a multi-network Hardhat project, managing private keys securely using environment variables, and interacting with a Layer 2 EVM-compatible blockchain. Polygon's low gas fees and fast block times make it a practical environment for deploying and testing decentralized applications.